

MAE 3260 Final Group Work: Satellite Attitude Control

Report

Joshua Baker, Richard Orellana, Maddox Nesterczuk, Parker Ashworth

Title: **satellite yay :)**

Topic of Interest: satellite attitude control

Abstract: Imaging satellites adjust their attitude to align imaging systems with targets of interest. This is achieved through attitude control systems, which apply torques to the spacecraft to cause changes to its orientation. Momentum actuators such as reaction wheels generate internal torques that reorient the spacecraft without changing the total angular momentum of the system. Thrusters provide external torques for rapid slew maneuvers but are reliant on propellant amount. This report examines the modeling and performance of an imaging satellite which uses these technologies.

Students/Roles:

Student	Task/Role	Portfolio
Joshua Baker	Sections 2-4, modeling and analytical solutions.	
Richard Orellana	I wrote the overview (section 1) and helped develop the earlier versions of the model.	Portfolio Link
Maddox Nesterczuk	Section 5, Advanced Applications. Created reaction wheel model and simulation code.	
Parker Ashworth	Developed model and assumptions, wrote abstract, and diagrammed proportional control and PD control.	

List of MAE 3260 concepts or skills used in this group work:

- P/PD Feedback Control
- State Space Representations
- Block Diagrams
- ODEs for System Dynamics
- Step Response and Steady-State Behavior

1: Overview

The system in question is an imaging satellite that is held in a fixed position in space. Initial analysis of the system was focused on different methods of attitude control and different attitude representations. The goal of this satellite system is to be capable of going from any arbitrary attitude to any other based on lump-sum control inputs in a body defined frame.

A toy planar model was developed to inspect the behavior of the system given no control inputs. The results of this planar model were expanded upon through the implementation of planar reaction wheels and planar thrusters. The implementation of active control systems, such as PI or PID, was not under consideration until the model was further developed. The reaction wheel planar model provided interesting dynamics to analyze as the torque produced by the wheels is internal to the system. In these planar models, attitude was represented with a simplified 2 by 2 direction cosine matrix with the angle measured counterclockwise from a predefined inertial +x axis.

Further development of the model pushed it into three dimensions. A small angle condition was applied to the model, reducing its imaging capabilities to a small cone centered about its initial pointing directions. This was absolutely necessary to maintain the linearity of the model. Certain cross product terms, such as ones involving the rotational rate of the satellite with itself, were neglected due to their relatively small magnitude. Control was once again applied through a lump-sum approach, that is to say that the directionality of the attitude control device was not considered. To better accommodate the introduction of a new dimension, quaternions replaced DCMs for attitude representation. This came with some challenges as it is much harder to visualize a three dimensional quaternion rotation than a two dimensional one.

Ultimately, it was decided that thrusters should be used as the control input devices. To sidestep the need for knowing the thrust application directions of each thruster, lumped control was once again used. The lessons learned from the planar model were helpful in developing planar control in three dimensions. A plane was defined with the cross product between the current pointing direction and the desired pointing direction as its normal. The torque direction required to accomplish the maneuver was along the normal of this plane. Not only is this model planar, it is also linear which greatly simplifies the calculations. We analyzed a version of this system assuming that proportional control with a step input was implemented. We then analyzed a version that implemented proportional and derivative control, tuning the gain constants to meet certain standards.

While a planar, linear model makes for simple calculations, nonlinear computer models usually allow for wider angle maneuvers. The dynamics behind reaction wheels are perfect for analyzing a non-linear version of the system, especially if quaternions are used for attitude control. Since this version of the model does not contain any constraints used in the previous versions, it also includes the application directions of the control input devices. This version of the system represents attitude using both quaternions and Euler angles.

2: Model and Assumptions

In reality, attitude control often involves the use of multiple reaction wheels or sets of thrusters to manipulate rotation in 3d space. However, this leads to complex, nonlinear dynamics. As the satellite rotates, so too do the directions in which a reaction wheel or set of thrusters can apply a torque. This nonlinearity makes attitude control algorithms difficult to develop or analyze. Therefore we choose to relegate the nonlinear dynamics to section 5) Advanced Applications, where we simulate attitude control with reaction wheels.

The bulk of our analysis will be on a simplified case with an externally applied torque in an arbitrary direction. We define the following variables and relationships:

- $\hat{r}$ Reference vector, desired pointing direction
- $\hat{p}$ Current pointing vector
- $\hat{\tau}$ Applied torque in direction $\frac{\hat{p} \times \hat{r}}{\|\hat{p} \times \hat{r}\|}$
- θ Angle between $\hat{r}$ and $\hat{p}$, defined by $\theta = \arccos(\hat{p} \cdot \hat{r})$
- J Satellite moment of inertia, $10^4 \text{ kg } m^2$
- $\tau = -J\ddot{\theta}$ ODE defining the system

Under this assumption, the problem becomes planar and linear. The motion is confined to the plane defined by $\hat{r}$ and $\hat{p}$, while the torque is applied normal to this plane. We also assume that the satellite is a cylinder whose central axis is defined by $\hat{p}$. Therefore, the problem is linear in θ and we do not need to consider variations in moment of inertia as the satellite rotates.

We will analyze two control schemes, proportional control and proportional/derivative control. For both controllers, we define necessary performance metrics:

- $|u(t)| < 50 \text{ Nm}$, for all time—defines the maximum output of the thrusters.
- $t_s < 300 \text{ s}$, where t_s is defined as the 99.8% settling time—maximum time to transition between and tolerance in angle. 99.8% is likely far too inaccurate for a space telescope, but it serves as a representative value.

3: Proportional Control

For the first controller, we will consider proportional control. Our control torque will be defined as $u(t) = K\theta$.

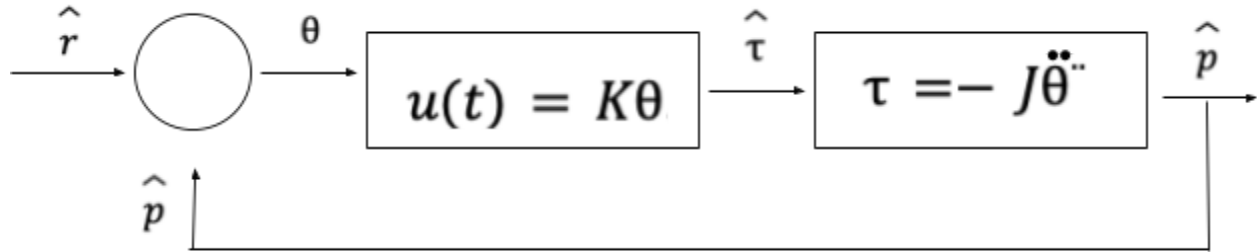


Figure 1: Block diagram of a proportional attitude control system. The measured angle θ is multiplied by the proportional gain K to generate the control torque $u(t)$, which is applied to the rigid body dynamics $\tau = -J\ddot{\theta}$

With this applied torque, the defining ODE becomes $J\ddot{\theta} + K\theta = 0$. However, we know that ODEs of this form are purely sinusoidal. Therefore, any attitude control scheme utilizing pure proportional control will oscillate indefinitely and never converge towards the target vector. This control system would only be viable in a system with damping. Instead, we move onto derivative control.

4: Proportional and Derivative Control

A more effective control scheme is including derivative control. This, in effect, will include a damping term that will allow us to converge to our desired direction more quickly. Our control effort becomes $u(t) = K_p \theta + K_d \dot{\theta}$, and our ODE becomes $J\ddot{\theta} + K_d \dot{\theta} + K_p \theta = 0$.

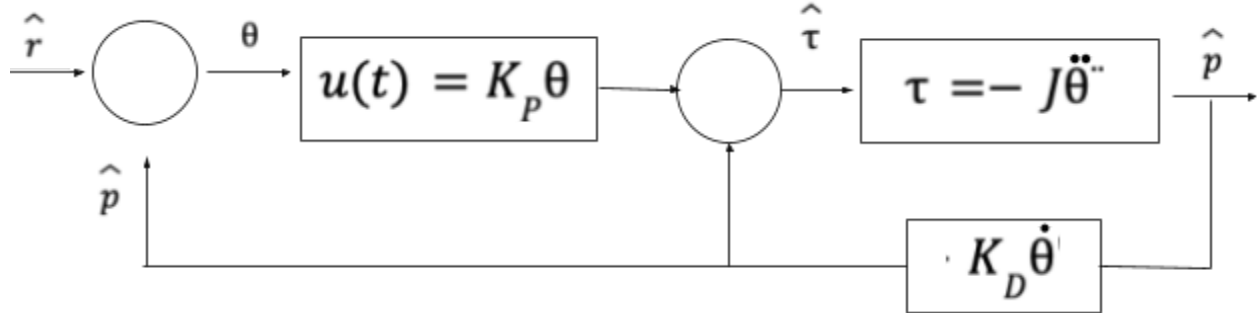


Figure 2: Block diagram of a proportional-derivative (PD) attitude control system. The controller uses both the angle θ and its rate $\dot{\theta}$ to generate the control effort $u(t) = K_p \theta + K_d \dot{\theta}$.

This is the a standard ODE of an unforced mass spring damper with $\omega_n = \sqrt{\frac{K_p}{J}}$ and $\zeta = \frac{K_d}{2\sqrt{JK_p}}$.

For the fastest settling, we choose a critically damped system where $\zeta = 1$, implying $4JK_p = K_d^2$. The general solution to this equation is $\theta(t) = (A + Bt)e^{-\omega_n t}$. Applying initial conditions of $\theta(0) = \theta_0$ and $\dot{\theta}(0) = 0$, we find that $\theta(t) = \theta_0(1 + \omega_n t)e^{-\omega_n t}$.

To choose K, we apply our two conditions.

Settling time:

$$.002 = (1 + \sqrt{\frac{K_p}{J}} t_s) e^{-\sqrt{\frac{K_p}{J}} t_s}$$

Unfortunately, this equation is transcendental, so we solve numerically to find

$$\sqrt{\frac{K_p}{J}} t_s = 8.462, \text{ so } t_s = 8.462 \sqrt{\frac{J}{K_p}} < t_{max} \text{ and thereby } K_p > \frac{71.6J}{t_{max}^2}.$$

Maximum control effort:

$$\begin{aligned} u(t) &= K_p \theta + K_d \dot{\theta} = K_p \theta_0 (1 + \omega_n t) e^{-\omega_n t} - 2\sqrt{JK_p} \theta_0 \omega_n^2 t e^{-\omega_n t} \\ &= \theta_0 [K_p (1 + \omega_n t) - 2\sqrt{JK_p} \omega_n^2 t] e^{-\omega_n t} \end{aligned}$$

This equation is also difficult to solve, but by graphing $u(t)$ for various K_p , we can observe that maximum control effort always occurs at $t = 0$. With this knowledge, our equation

simplifies to $u_{max} = K_p \theta$, leading us to conclude that $K_p < \frac{u_{max}}{\pi}$, since the maximum value of θ is π .

Therefore, $\frac{71.6J}{t_{max}^2} < K_p < \frac{u_{max}}{\pi}$. Plugging in values, we find $7.96 Nm < K_p < 15.92 Nm$ while $K_D = 200\sqrt{K_p}$. One possible selection is $K_p = 8 Nm$ and $K_D = 566 Nms$.

5: Advanced Applications

So far, torques have been imparted onto the spacecraft externally via thrusters. Momentum actuator technologies are used to change the direction of the angular momentum vector of a spacecraft by introducing internal torques, thereby changing the system's orientation in the inertial frame. Common momentum actuator technologies include reaction wheels and control-moment gyroscopes. This section will involve an investigation into attitude control utilizing reaction wheels.

Symbols

$\mathbf{v}$ — Vector denoted with boldface type

$\dot{\mathbf{v}}$ — Time derivative of vector

$\mathbf{A}^{-1}$ — Matrix inverse

$\mathbf{A}^T$ — Matrix transpose

$\mathbf{A}^+$ — Matrix pseudo-inverse

$\mathbf{A}$ — Jacobian to transform from torque inputs to momentum derivative of reaction wheels

$\boldsymbol{\omega}^{B/N}$ — Angular velocity of B (body-fixed) frame with respect to N (inertial) frame

$\mathbf{I}^B$ — Inertia matrix expressed in B (body-fixed) frame

$\dot{s}$ — Derivative of scalar

$\mathbf{H}^N$ — Time derivative of net angular momentum about the center of mass in N (inertial) frame

$\boldsymbol{\omega}^{B/N}$ (**B-frame derivative**) — B (body-fixed) frame time derivative of B (body-fixed) frame's angular velocity with respect to the N (inertial) frame

$\mathbf{h}^B$ — Net angular momentum of momentum actuator technologies

τ_i^{ext} — i^{th} external torque

$\mathbf{a}_i$ — i^{th} unit vector

$\mathbf{q}$ — Quaternion

$\dot{\mathbf{q}}$ — Quaternion derivative

5.1 System Definition

Our system includes the spacecraft and its momentum actuator technologies. Because these actuators are internal, all torques they generate are internal torques. External torques (e.g., thruster forces) are not modeled.

5.2 Equation of Motion

The total angular momentum in the body-fixed frame is:

$$\mathbf{H}^B = \mathbf{I}^B \cdot \boldsymbol{\omega}^{B/N} + \mathbf{h}^B$$

Taking the derivative and applying the transport theorem yields the equation of motion:

$$\mathbf{H}^N = \mathbf{I}^B \cdot \boldsymbol{\omega}^{B/N} + \boldsymbol{\omega}^{B/N} \times (\mathbf{I}^B \cdot \boldsymbol{\omega}^{B/N} + \mathbf{h}^B) + \mathbf{h}^B$$

Euler's Second Law states:

$$\mathbf{H}^N = \sum \mathbf{T}_i^{\text{ext}}$$

Since there are no external torques:

$$\sum \mathbf{T}_i^{\text{ext}} = \mathbf{H}^N = 0$$

Solving for $\boldsymbol{\omega}^{B/N}$ gives:

$$\boldsymbol{\omega}^{B/N} = (\mathbf{I}^B)^{-1} [\boldsymbol{\omega}^{B/N} \times (\mathbf{I}^B \cdot \boldsymbol{\omega}^{B/N} + \mathbf{h}^B) + \mathbf{h}^B]$$

Integrating:

$$\boldsymbol{\omega}^{B/N}(t) = \int_0^t \boldsymbol{\omega}^{B/N} d\tau$$

5.3 Control Term

The actuator angular-momentum derivative $\dot{\mathbf{h}}^B$ is controlled by imparting reaction wheel torques:

$$\dot{\mathbf{h}}^B = \sum T_i \hat{\mathbf{a}}_i$$

To solve for $\dot{\mathbf{h}}$ components:

$$\begin{aligned} \dot{\mathbf{h}} &= \mathbf{A}^+ \mathbf{T}, \\ \mathbf{A}^+ &= \mathbf{A}^T (\mathbf{A} \mathbf{A}^T)^{-1} \end{aligned}$$

5.4 Quaternion Representation

A quaternion representing orientation is:

$$\mathbf{q} = [q_0; q_1; q_2; q_3]$$

Quaternions can be converted to Euler angles or direction cosine matrices (DCMs).

5.5 Orientation Time History

The quaternion derivative is computed from angular velocity:

$$\dot{\mathbf{q}} = (\boldsymbol{\omega}^{B/N} ; 0) \otimes \mathbf{q}_b$$

Integrating:

$$\dot{q}(t) = \int_0^t \dot{q}_b \, d\tau$$

5.6 Summary: From Input Torque to Orientation

1. A torque is inputted into the reaction wheels by a controller. Components of this torque are transformed into components of the inertial derivative of angular momentum.
2. The equation of motion is integrated to find the angular velocity time history of the body-fixed frame
3. The angular velocity is used to calculate the quaternion derivative, which is integrated to obtain the quaternion representing spacecraft orientation. This quaternion can be converted into a DCM.

5.7 Simulink Model

We used simulink to model the dynamics described above. The simulink model takes step input torques, representing reaction wheels oriented along the body fixed axes in the spacecraft, and outputs the quaternion that describes the spacecraft's orientation. The model is also capable of visualizing the evolution of the spacecraft's orientation.

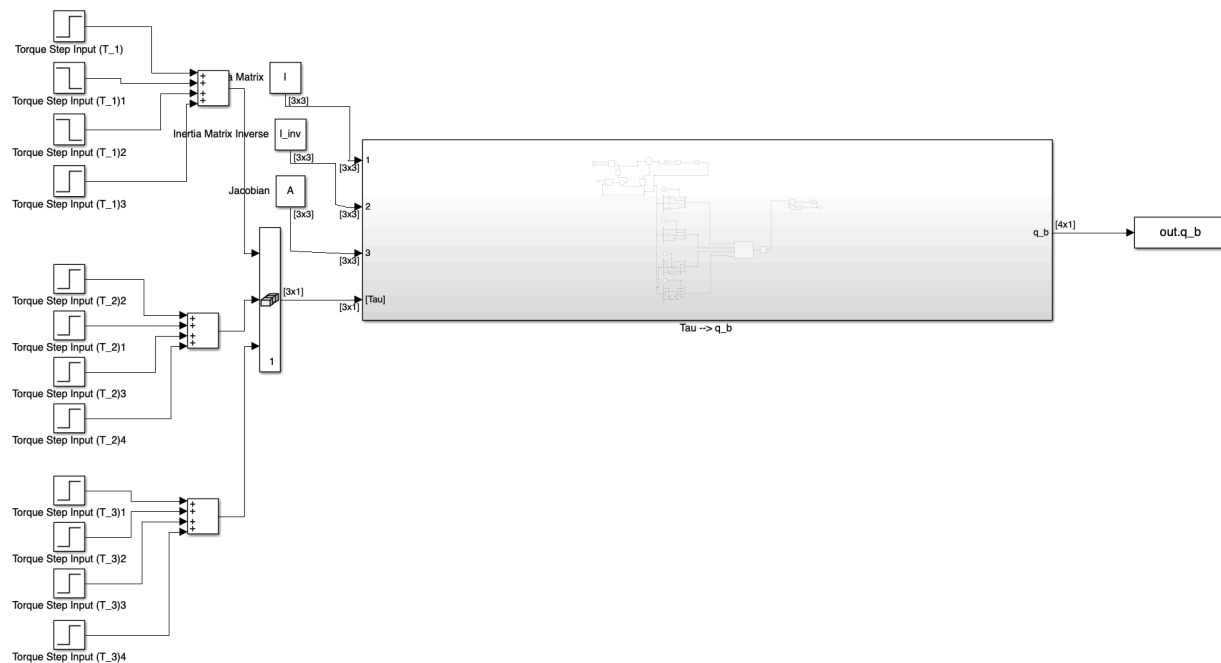


Figure 1. Complete simulink Model for attitude control through reaction wheels.

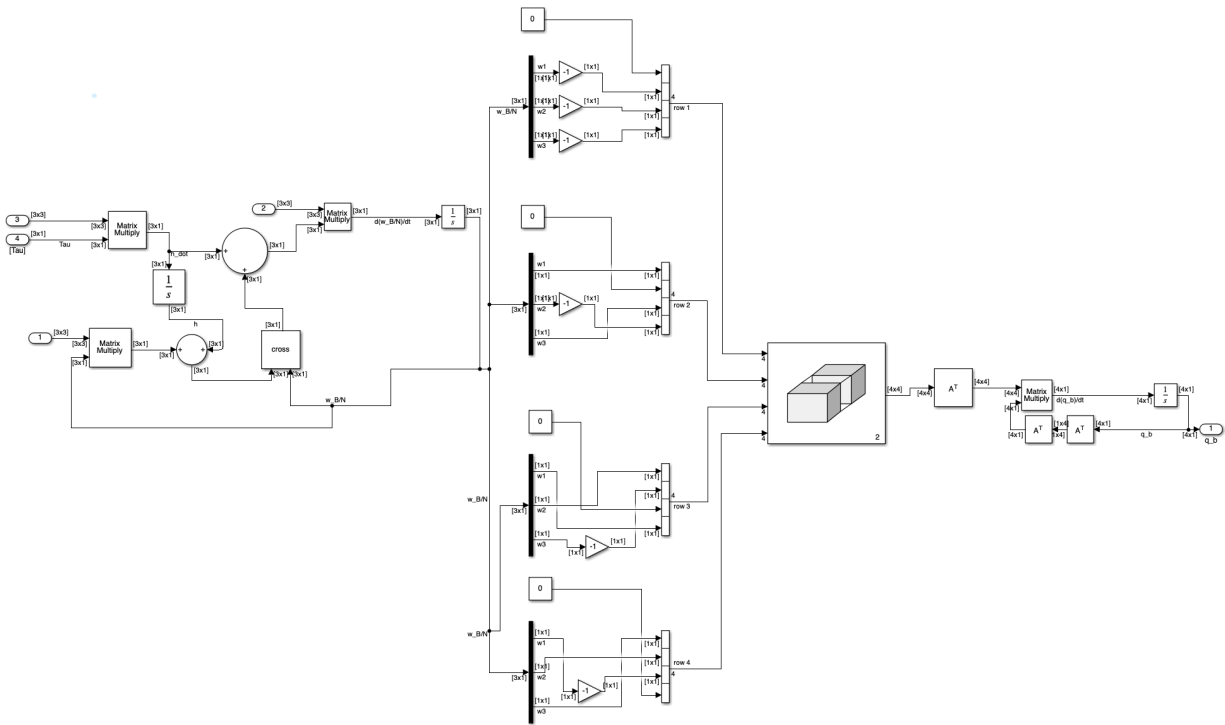


Figure 2. Attitude dynamics block to convert torque inputs into quaternion attitude state.

```

1  model = 'Attitude_Control_System';
2  T_stop = 10;
3  open_system(model);
4  set_param(model, 'StopTime', num2str(T_stop));
5
6  out = sim(model);
7
8  quat_ts = out.q_b;
9  time = quat_ts.Time;
10 q = quat_ts.Data;
11
12 L = 1; W = 1; H = 1;
13 verts = [ L  W  H;
14          -L  W  H;
15          -L -W  H;
16           L -W  H;
17           L  W -H;
18          -L  W -H;
19          -L -W -H;
20           L -W -H ];
21
22 faces = [1 2 3 4;
23          5 6 7 8;
24          1 5 8 4;
25          2 6 7 3;
26          1 5 6 2;
27          4 8 7 3];
28
29 figure(1);
30 h_patch = patch('Vertices', verts, 'Faces', faces, ...
31                'FaceColor', [0.2 0.5 1], 'FaceAlpha', 1);
32 axis equal; grid on;
33 xlabel('X'); ylabel('Y'); zlabel('Z');
34 view(3);
35 title('Spacecraft Attitude Animation');
36
37 bound = 3;
38 xlim([-bound, bound]);
39 ylim([-bound, bound]);
40 zlim([-bound, bound]);
41
42
43 playback_speed = 1;
44 for k = 1:length(time)
45     if k == length(time)
46         else
47             pause((-time(k) + time(k+1))/playback_speed)
48         end
49         qk = (q(:,1,k))';
50         qk1 = qk(1);
51         qk2 = qk(2);
52         qk3 = qk(3);
53         qk4 = qk(4);
54         q_in = [qk4, qk1, qk2, qk3];
55         R = quat2rotm(q_in);
56
57         rotated_verts = (R * verts)';
58
59         set(findobj(gca,'Type','patch'), 'Vertices', rotated_verts);
60
61         drawnow;
62     end
63

```

Figure 3. Code to run simulink model and generate spacecraft animation. Current settings for the torque inputs cause spacecraft to slew about z-axis for 10 seconds.

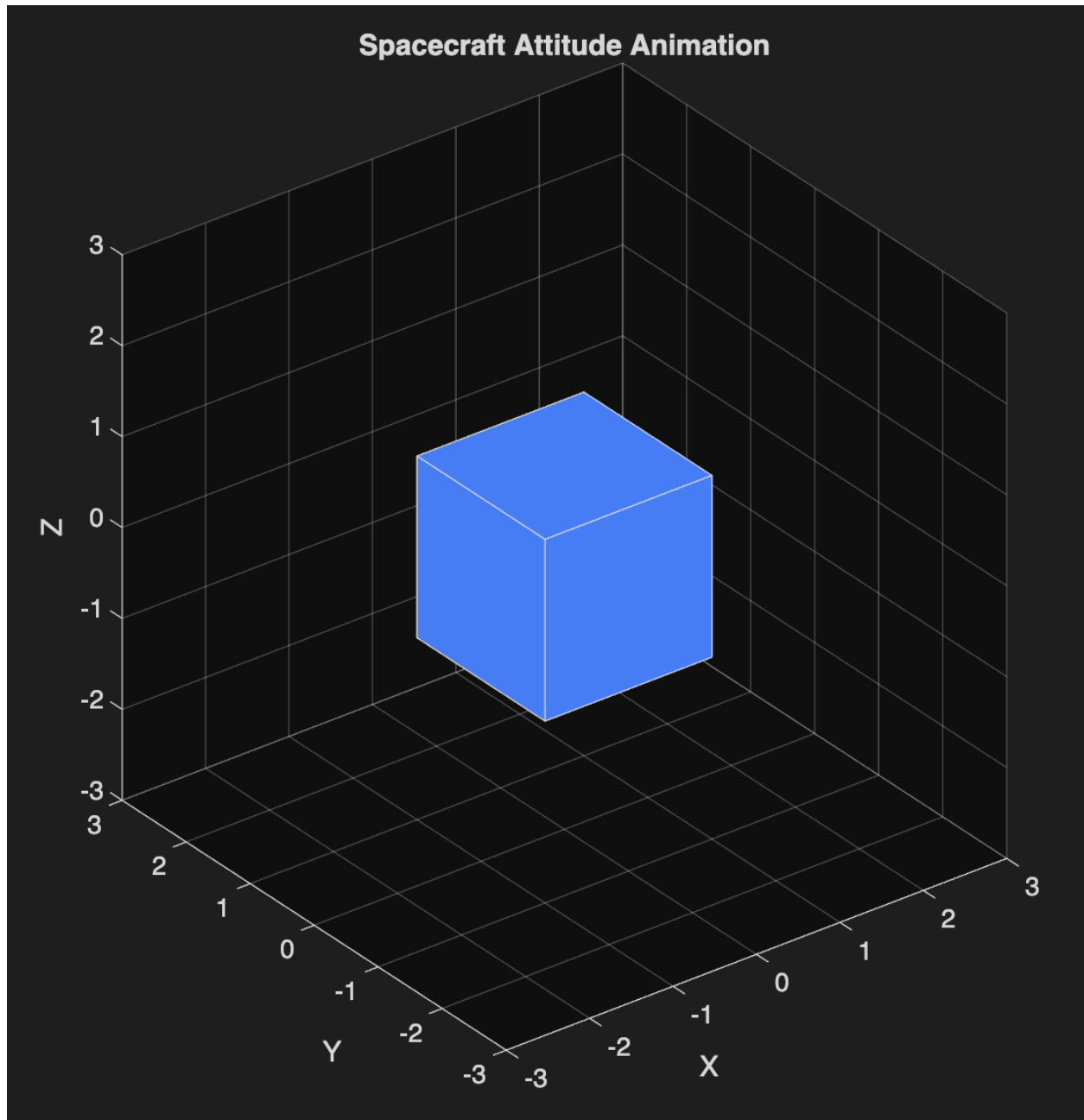


Figure 4. Spacecraft orientation before slew about the z-axis.

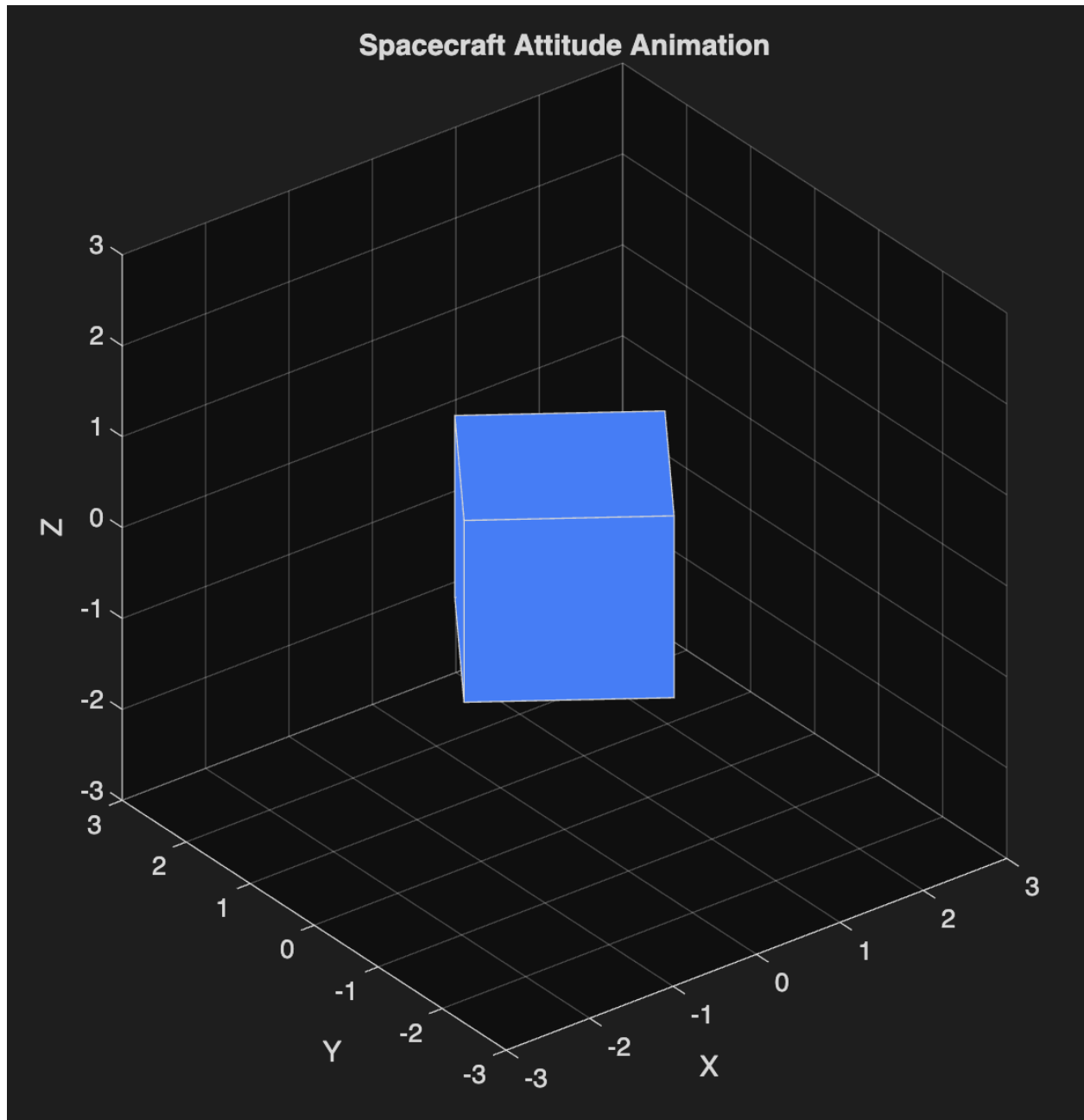


Figure 5. Spacecraft orientation after slew about the z-axis.